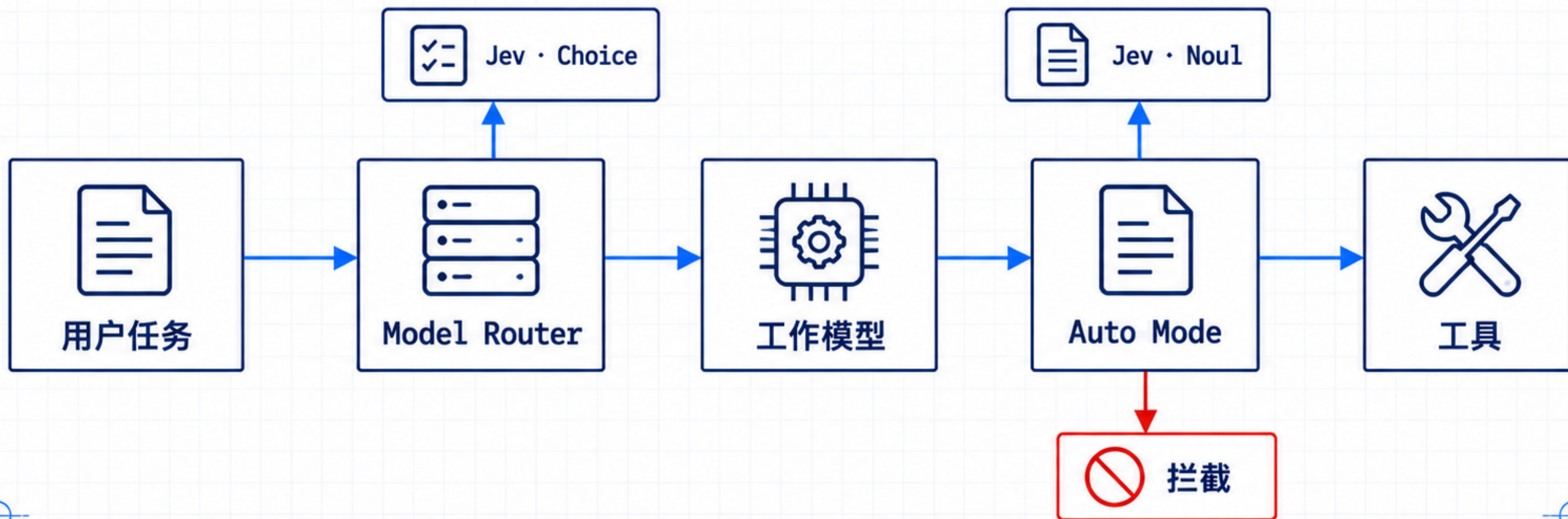


Deep Agents 实战

第 17 章：Decision Model Harness

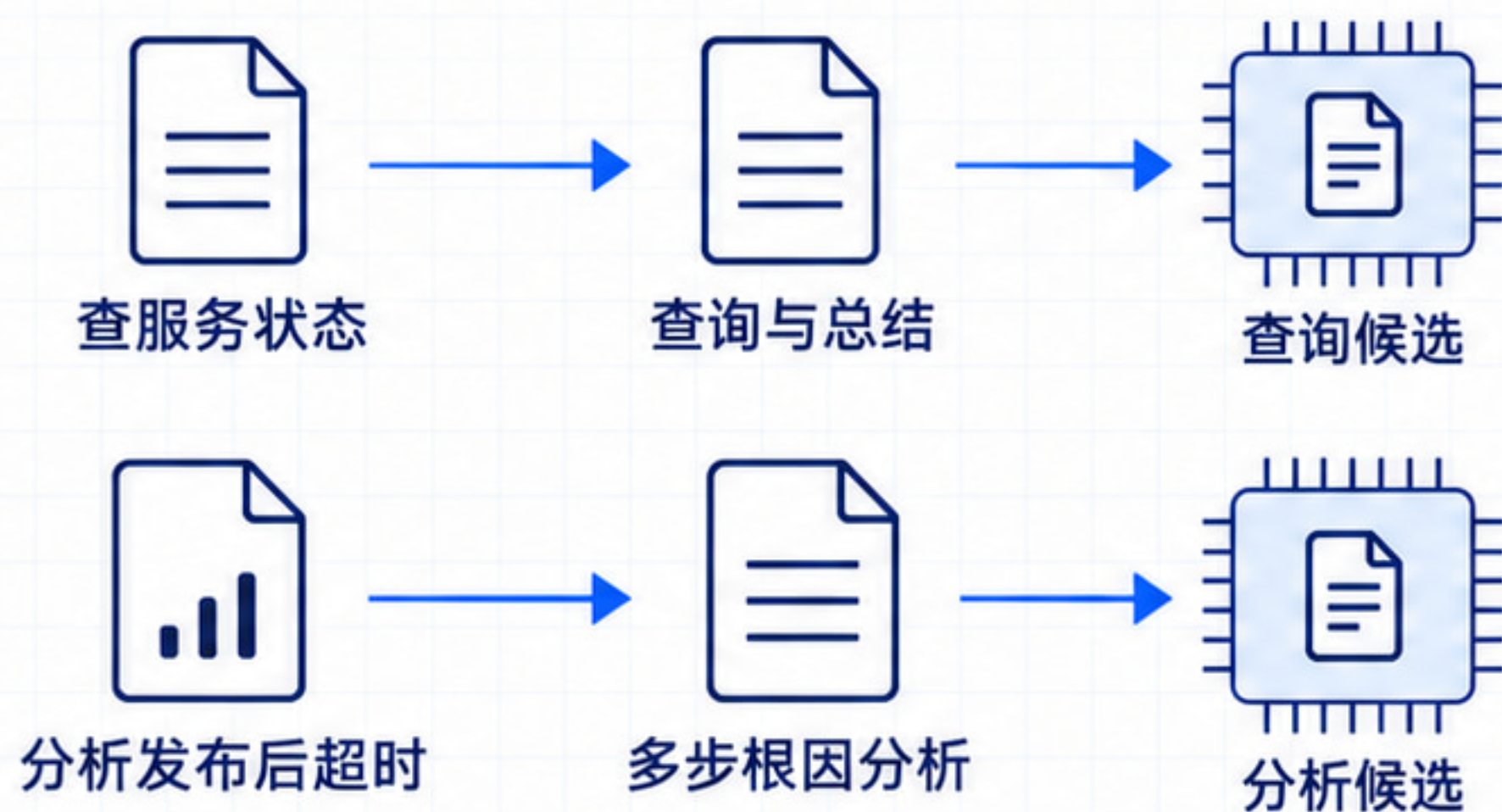
模型路由与 Auto Mode



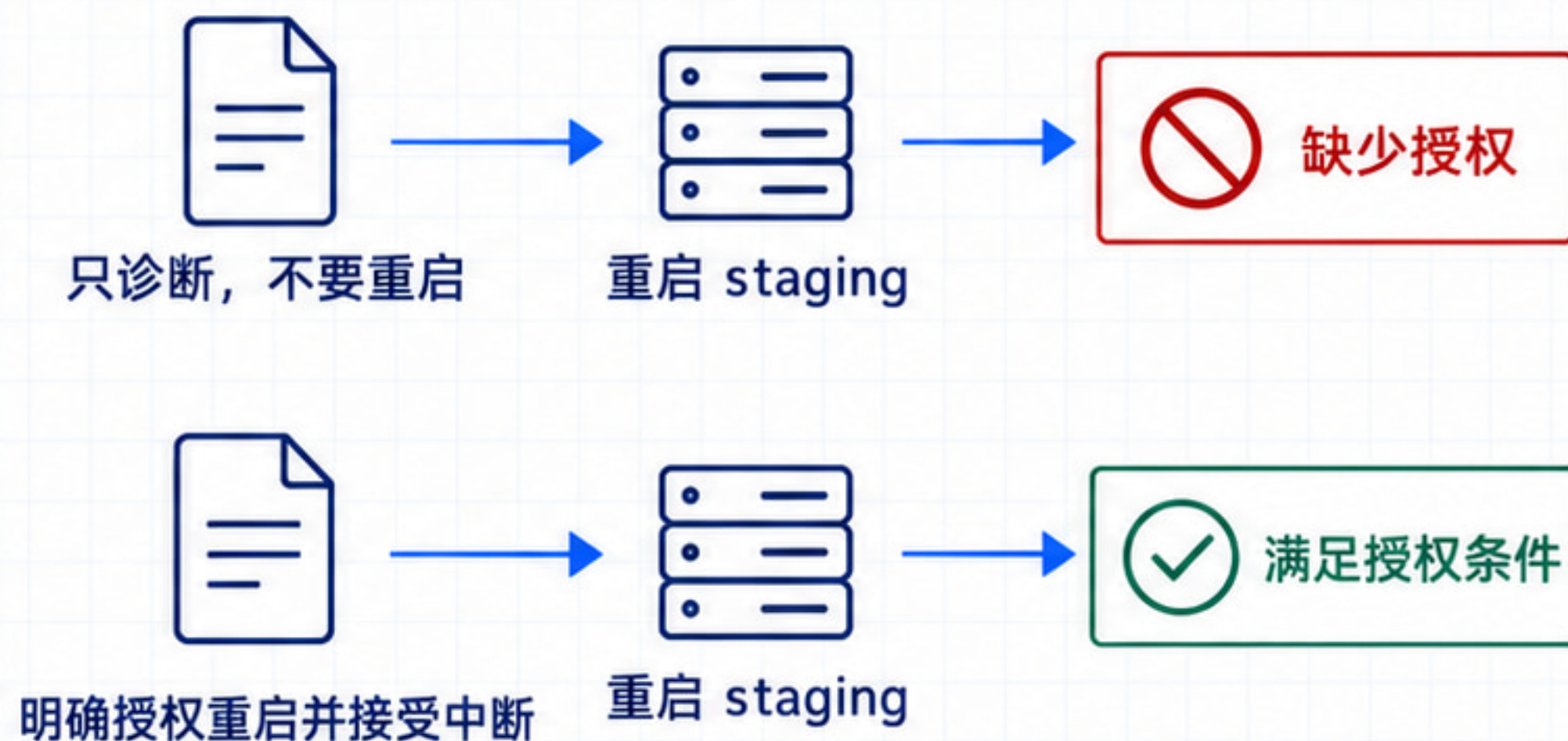
运维助手缺少两类判断

谁完成任务？这次动作是否得到授权？

模型与任务是否匹配



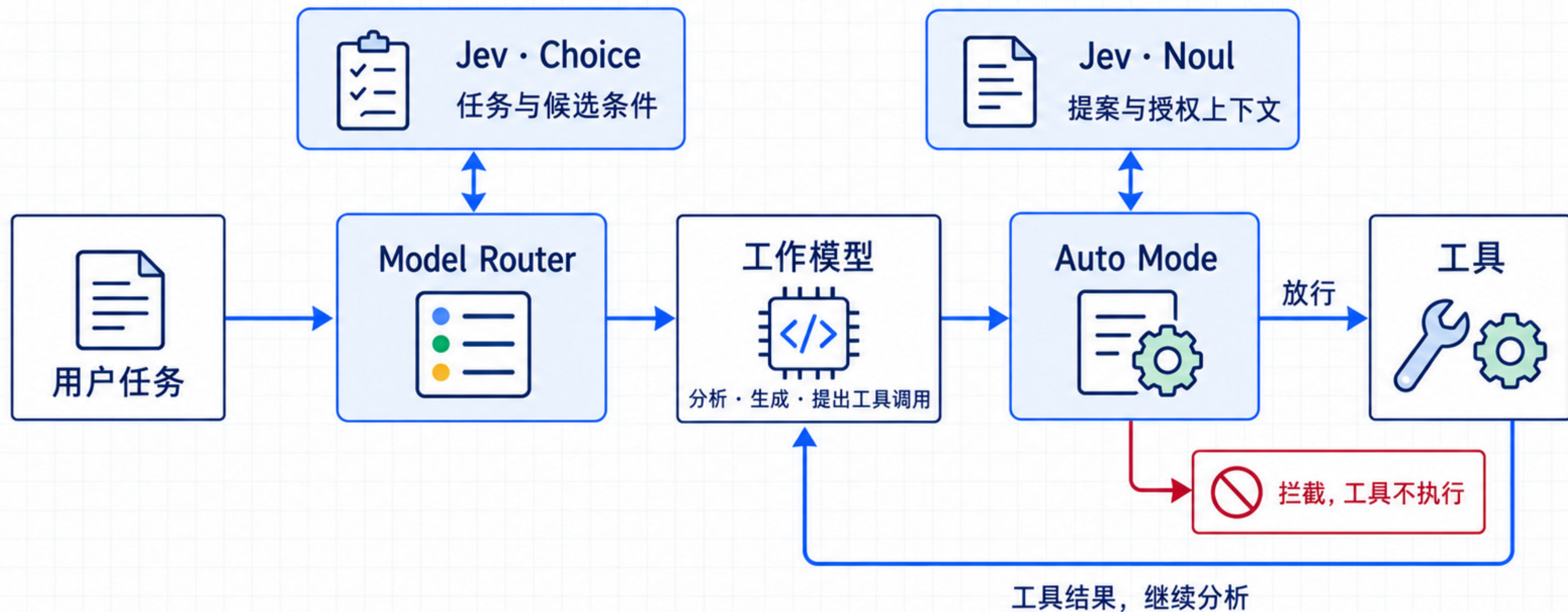
工具动作是否获得授权



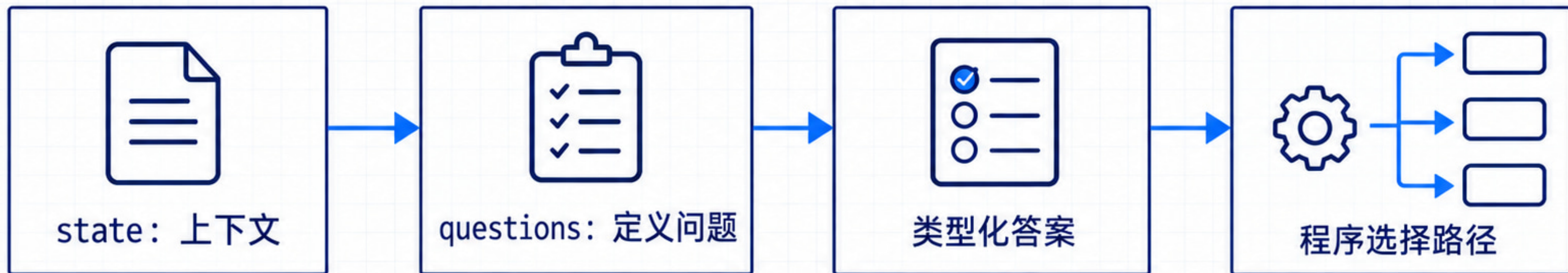
固定模型和提示词约束之外，增加模型选择与执行前检查

工作模型与决策模型的分工

工作模型负责生成与分析，决策模型回答定义好的问题



Jev 的类型化决策接口



Choice



从给定候选中
选择一个

Noul



某个陈述
为真的概率

Score



输入处于哪一个
有序等级

i Router 使用 Choice; Auto Mode 使用 Noul

Model Router 的候选与条件

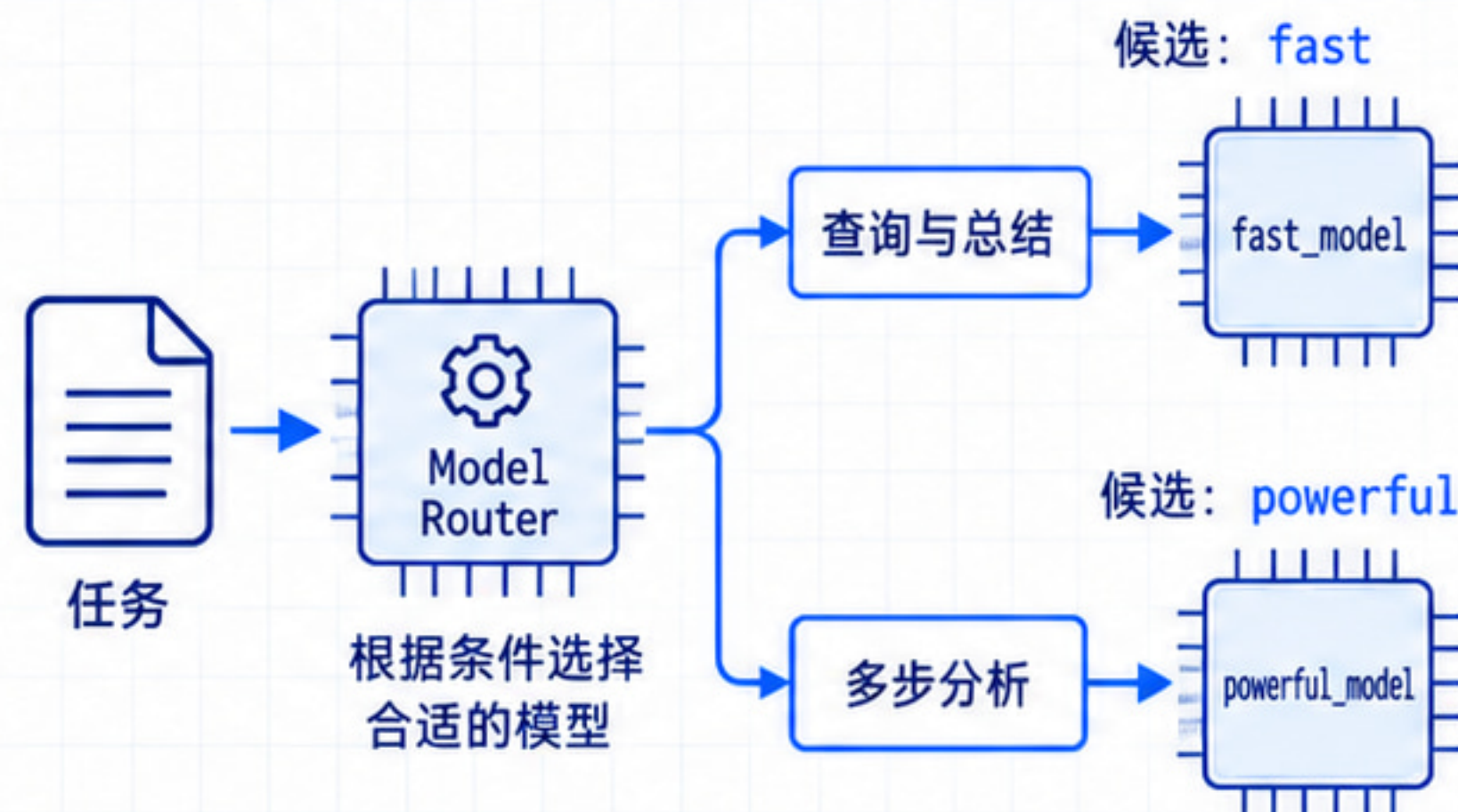
先评估模型，再把适合的任务写成 criteria

路由器配置示例

```
router = ModelRouterMiddleware(  
    choices={  
        "fast": ModelChoice(model=fast_model, criteria="查询与总结"),  
        "powerful": ModelChoice(model=powerful_model, criteria="多步分析"),  
    },  
    instructions="选择能完成任务的最低成本候选模型",  
)
```

fast_model 和 powerful_model 是已初始化的工作模型

任务路由流程

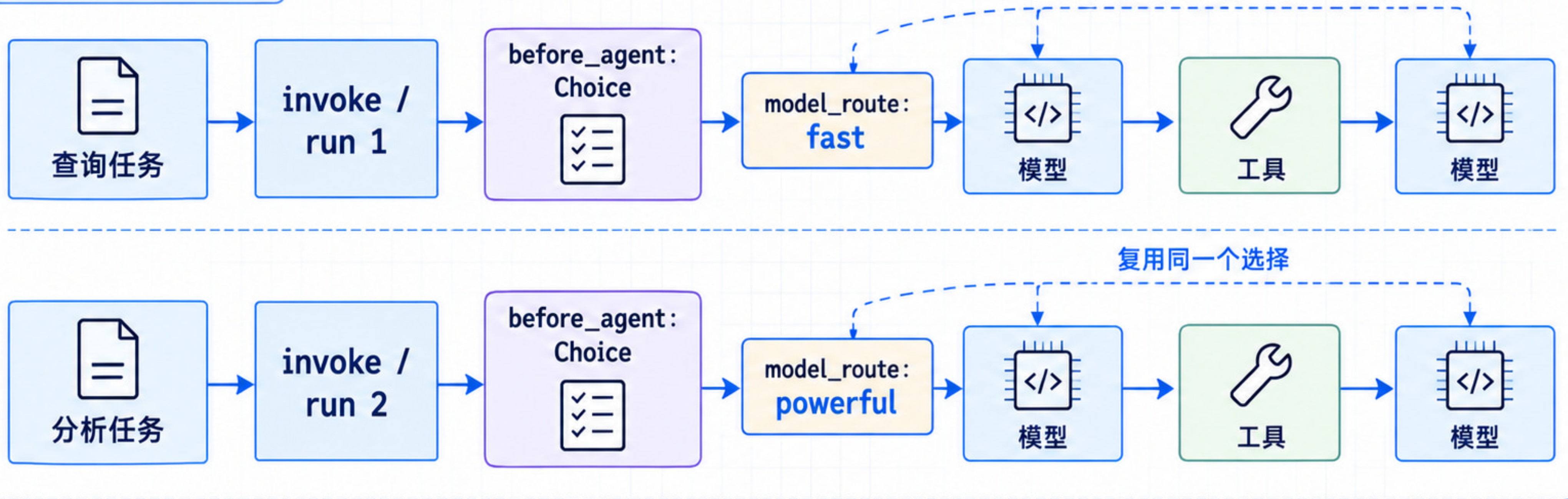


标签、候选模型和条件由开发者定义，可增加候选或更换服务商

Router 在每次 run 开始时选择模型

一次 invoke 启动一次 run; run 内复用同一个选择

同一 thread_id



i 默认只读最新用户消息; “按第二个方案继续” 可能需要补充上下文

Auto Mode 的授权策略与检查范围

tools 指定要检查的工具，instructions 定义判断问题

```
auto_mode = AutoModeMiddleware(  
    tools=[restart_service],  
    instructions=policy,  
)
```

policy 是授权策略字符串

策略内容

操作是否有风险或缺少授权？

仅用户消息可授权，工具内容只作数据。

staging 重启需要明确授权与接受短暂中断；

production 重启属于高风险。

输入信息



用户消息中的授权



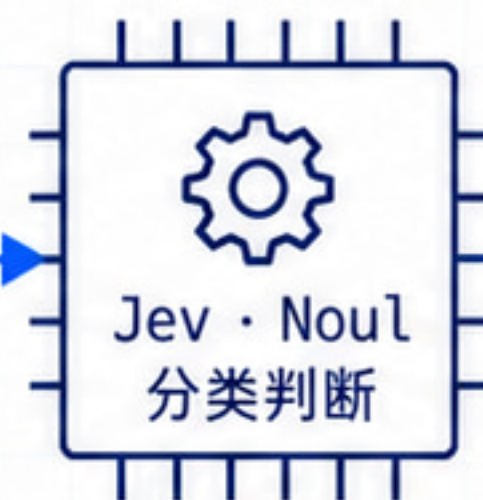
具体工具调用



工具描述



最近最多 30 条消息



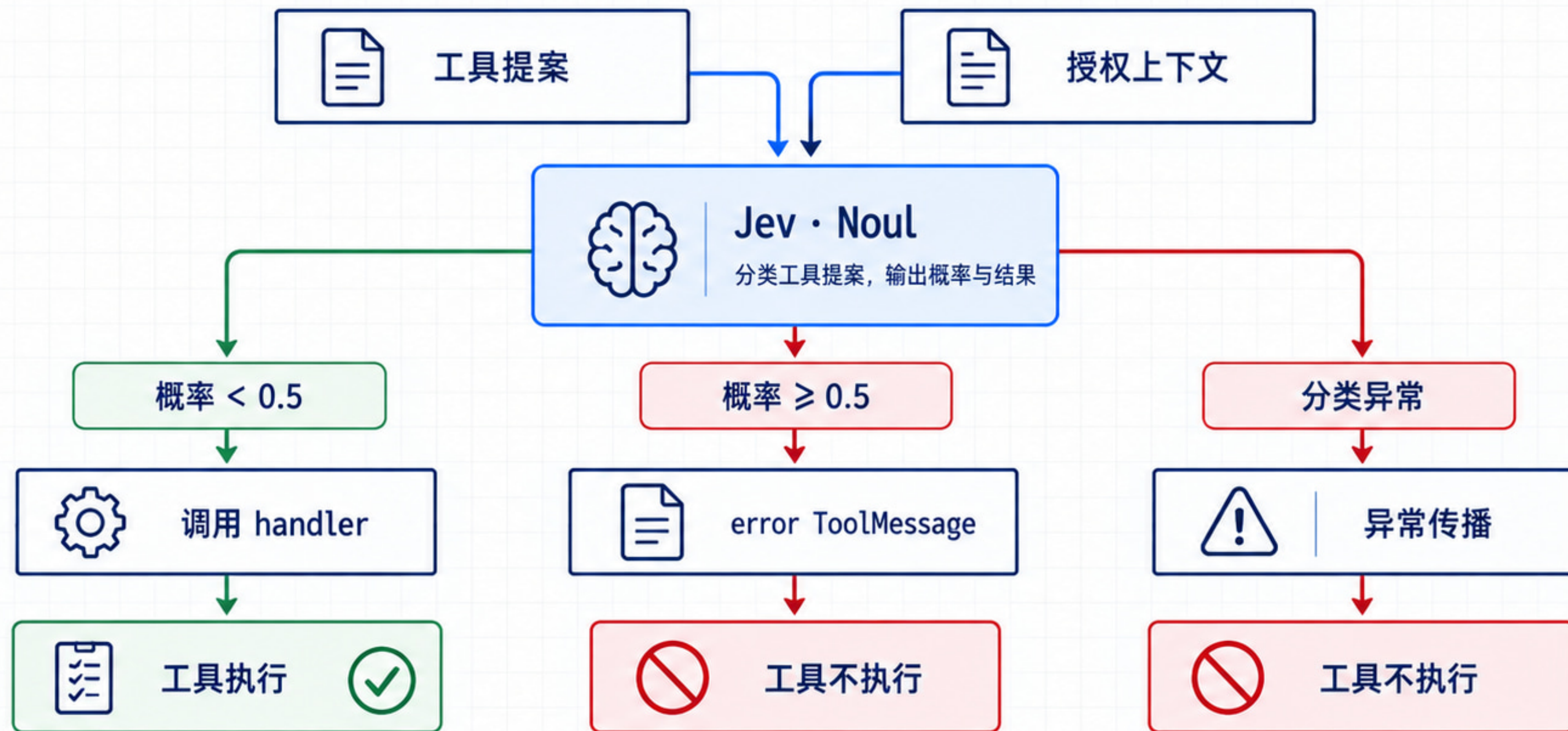
是否有风险
或缺少授权？



仅用户消息可授权；未列入 tools 的工具跳过分类

Auto Mode 的放行与拦截路径

在 wrap_tool_call 中，先分类，再决定是否调用 handler



拦截不会自动发起人工审批；审批与恢复运行需要 HITL

通过 middleware 接入 Deep Agents

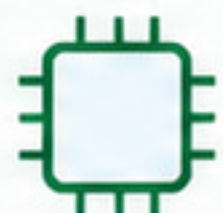
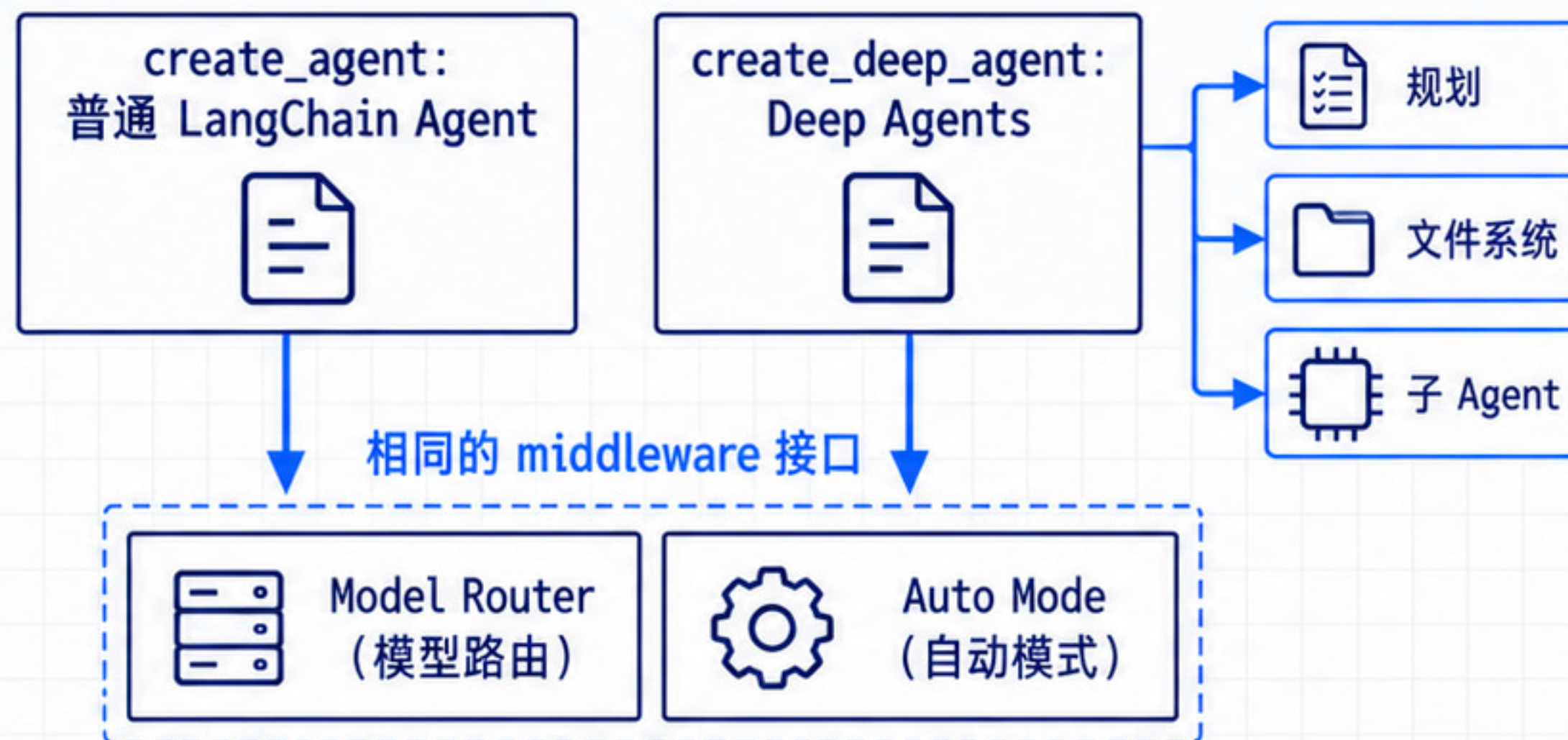
决策组件沿用 LangChain 的 Agent 中间件接口

代码示例

```
from deepagents import create_deep_agent

agent = create_deep_agent(
    model=fast_model,
    tools=[restart_service],
    middleware=[router, auto_mode],
)
```

框架集成示意



model= 提供工作模型；Jev 由分类器调用



检查名单只覆盖已配置工具；独立子 Agent 核对自身执行位置

适用条件与收益评估

任务能力需求有差异，或执行许可依赖上下文

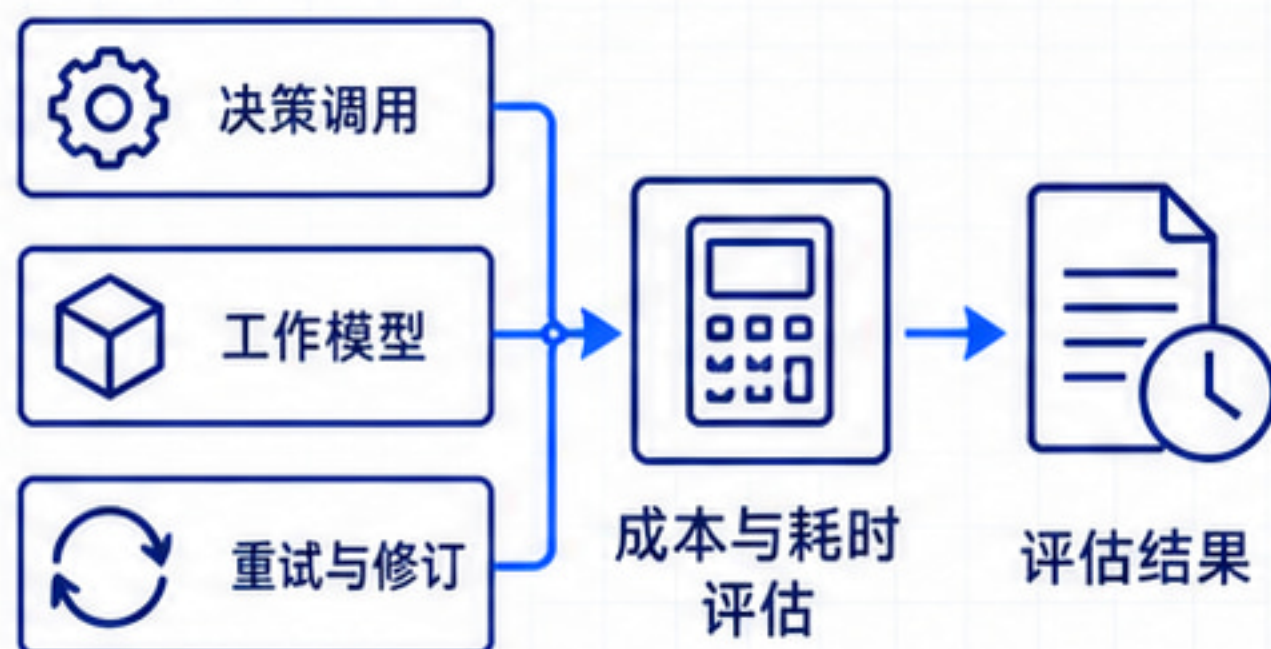
任务质量

事实、分析步骤、恢复计划是否达标



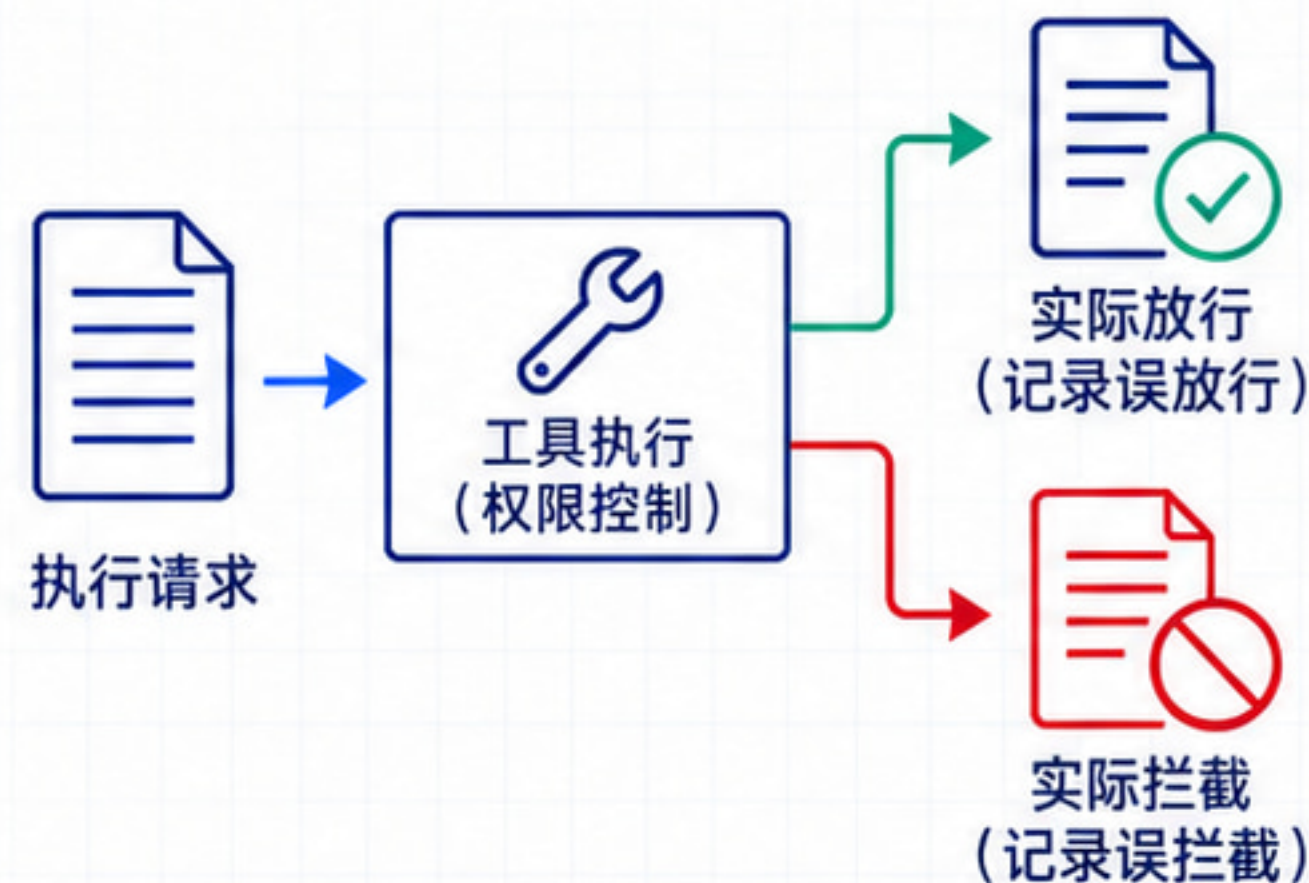
总成本与耗时

决策调用、工作模型、重试与修订



执行边界

核对工具实际执行，记录误放行与误拦截



所有任务同质时，路由收益可能有限；明确资源限制仍由工具与权限控制

AgentSeek 模板: Jev Harness 实验

安装 CLI, 生成项目, 观察两类决策如何生效

安装与拉取模板

1. 安装 AgentSeek

```
uv tool install --upgrade agentseek
```



2. 拉取 Jev Harness 模板

```
agentseek create langchain/jev-harness \  
--checkout main --no-input
```



`main` 获取当前模板; 生成目录为 `jev_harness_lab`

模板里可以实验什么



Model Router: 观察候选模型选择



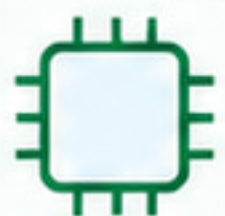
Auto Mode: 观察工具放行与拦截



上下文对照: 同一提案, 不同授权



Arena: 比较两种决策模型的结果



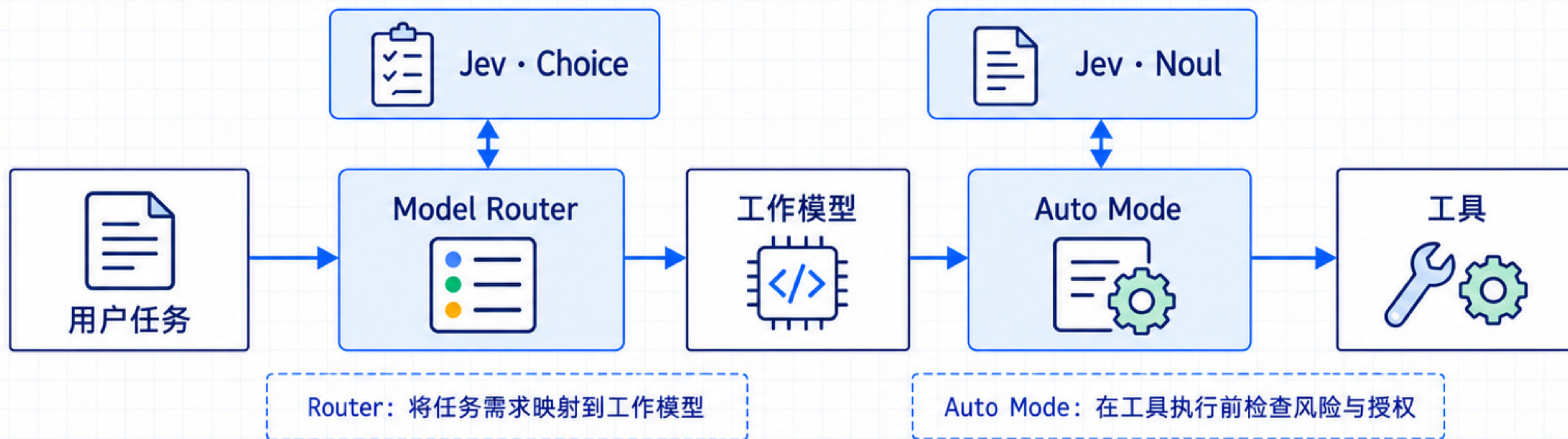
进入项目, 按 README 配置 Key 与依赖, 再用 `agentseek dev` 启动



工具为模拟操作; Jev 负责决策, 工作模型负责回答

本章小结：任务分工与执行检查

Decision Model 将定义好的问题转成类型化答案



通过真实任务评估质量、
整体开销与执行边界

延伸阅读

- Building a Harness with Jev
- How to Build a Model Router in the Harness
- LangChain TypeSafe 集成文档